UNIVERSIDADE FEDERAL FLUMINENSE
Pró-Reitoria de Pesquisa, Pós-Graduação e Inovação
Agência de Inovação — AGIR

RELATÓRIO FINAL

PIBITI/CNPq — PIBINOVA/PDI/UFF — 2025/2026

Diagrama Dinâmico de Fluxo Energético (DDFE)

Geração automática de Diagramas de Fluxo de Energia de Referência (RES) a partir de dados tabulares, em Power BI

Orientador: Tiago Abud
Bolsista: Frederico José Poli Antunes
Vigência: setembro de 2025 a setembro de 2026

Niterói — RJ
2026

Resumo

Modelos de planejamento energético de longo prazo — como os construídos em OSeMOSYS, MESSAGE ou TIMES — são descritos por um Diagrama de Fluxo de Energia de Referência (*Reference Energy System*, RES): um grafo dirigido em que combustíveis e vetores energéticos são conectados por tecnologias de extração, conversão, transporte e uso final. Esse diagrama é, na prática, a especificação visual do modelo. Na prática corrente ele é desenhado à mão em ferramentas de diagramação genéricas, resultando em um artefato caro de produzir, que desatualiza a cada alteração do modelo e não pode ser auditado nem interrogado. Existe geração automática fora do ambiente de *business intelligence* (BI), notadamente o comando `otoole viz res`, que produz o diagrama via Graphviz; ela resolve o custo de produção, mas entrega uma imagem estática cujas colunas são inferidas pela topologia do grafo, e não pelo estágio energético declarado nos dados.

Este projeto desenvolveu o Diagrama Dinâmico de Fluxo Energético (DDFE): uma ferramenta que gera automaticamente o diagrama RES completo — posições, roteamento de linhas, cores e interatividade — a partir de duas tabelas que o próprio modelador já mantém. Não há nenhuma coordenada desenhada manualmente. A solução é implementada sobre a pilha Excel → Power Query (linguagem M) → Deneb (Vega v5), dentro do Microsoft Power BI, e divide-se em duas camadas: um *back-end* geométrico em M, que resolve o problema de *layout* de grafo em camadas sob restrição semântica de camada; e um *front-end* declarativo em Vega, que renderiza as coordenadas pré-calculadas e implementa destaque interativo. As contribuições técnicas centrais são a alocação do eixo horizontal por *look-ahead* de densidade, a alocação do eixo vertical por raias elásticas com nós fantasmas, e o roteamento explícito de fluxos retroalimentados.

A ferramenta foi validada sobre um sistema energético nacional estilizado com 70 combustíveis em 5 estágios e 101 tecnologias, além de uma bateria de testes de esforço com topologias aleatórias. O resultado é genérico: qualquer base relacional que siga a matriz origem–destino padronizada é renderizada sem reconfiguração.

Palavras-chave: sistema energético de referência; visualização de dados; desenho de grafos em camadas; Power BI; OSeMOSYS.

Sumário

1 Diagnóstico 1
1.1 O Diagrama de Fluxo de Energia de Referência 1
1.2 Custo de produção e de manutenção 1
1.3 Estado da arte: o que já existe e onde para 2
1.4 Limitações das visualizações de plataformas de BI 2
1.5 Formulação do problema técnico 3
2 Soluções Propostas 4
2.1 Arquitetura em duas camadas 4
2.2 Pipeline de transformação 5
2.3 Algoritmos de layout 5
2.3.1 Eixo horizontal dos combustíveis: look-ahead de densidade 5
2.3.2 Eixo horizontal das tecnologias: paredes geométricas e fluxos reversos 6
2.3.3 Eixo vertical: raias elásticas e nós fantasmas 7
2.3.4 Lógica de hub e herança de cor 8
2.4 Camada de interação 9
2.5 Motor de rastreabilidade 11
2.6 Engenharia de processo e validação 12
3 Resultados Alcançados 12
3.1 Produto entregue 12
3.2 Contribuições técnicas 14
3.3 Produção documental 14
3.4 Dificuldades encontradas 15
3.5 Trabalhos futuros 15
3.6 Conclusão 16
Agradecimentos 16
Declaração de uso de Inteligência Artificial Generativa 16
Referências 17

Lista de Figuras

Figura 1 Especificação visual do sistema energético, desenhada manualmente em ferramenta de diagramação. 1
Figura 2 Estrutura de entrada do sistema. Em (b), as três linhas destacadas declaram as três saídas da tecnologia `PROETA1`. 4
Figura 3 Grafo de dependências entre as nove consultas do *pipeline*. 5
Figura 4 Roteamento automático de fluxos reversos nas termelétricas `PWRHFO`, `PWRNGS1` e `PWRNGS2`. 6
Figura 5 Algoritmo de raias elásticas com nós fantasmas. 8
Figura 6 Estado íntegro, sem seleção ativa. 10
Figura 7 Seleção do combustível `BIO`: a cadeia da biomassa é isolada. 10
Figura 8 Seleção do combustível `COA`: a cadeia do carvão é isolada e a da biomassa, atenuada. 11
Figura 9 Panorama do diagrama gerado automaticamente, abrangendo os combustíveis primários e secundários e parte das rotas de conversão. Reproduzido em escala reduzida para evidenciar a extensão do traçado. 13

Lista de Tabelas

Tabela 1 Divisão de responsabilidades entre as duas camadas da solução. 4
Tabela 2 Sinais reativos que implementam o destaque interativo. 9
Tabela 3 Características do sistema energético usado na validação da ferramenta. . . . 13

1 Diagnóstico

1.1 O Diagrama de Fluxo de Energia de Referência

Ferramentas de modelagem energética de sistemas representam o setor energético de um país ou região como uma rede de fluxos. A formulação canônica do OSeMOSYS (HOWELLS et al., 2011) e das famílias MARKAL/TIMES (LOULOU et al., 2005) parte da mesma abstração: combustíveis (*fuels* ou *commodities*) e tecnologias (*technologies*) que consomem um conjunto de combustíveis de entrada e produzem um conjunto de saída. Os combustíveis são organizados por estágio da cadeia energética — recurso primário, secundário, final, útil, demanda — e essa ordenação define a leitura do diagrama da esquerda para a direita.

O RES não é ilustração: é o contrato de especificação entre o modelador, o orientador, o revisor e o tomador de decisão. É por ele que se verifica se uma rota tecnológica foi esquecida, se um combustível ficou órfão — produzido e nunca consumido —, se há um ciclo indevido, ou se a mesma demanda está sendo atendida por duas cadeias concorrentes.

1.2 Custo de produção e de manutenção

Um RES de porte realista, como o adotado neste trabalho, tem da ordem de 170 nós e mais de 100 arestas. Desenhado à mão, consome dias de trabalho. O próprio projeto produziu essa evidência: a especificação visual que serviu de alvo, reproduzida na Figura 1, precisou ser desenhada caixa a caixa em ferramenta de diagramação antes que a automação existisse.

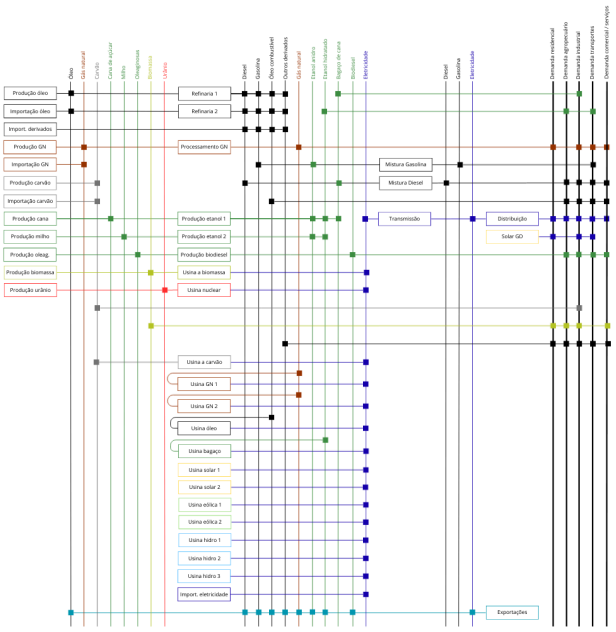


Figura 1 — Especificação visual do sistema energético, desenhada manualmente em ferramenta de diagramação.

O problema mais grave, contudo, não é o custo da primeira produção, e sim o da manutenção. Toda alteração estrutural do modelo invalida o desenho: acrescentar um único

combustível intermediário obriga a reposicionar dezenas de caixas e a redesenhar as linhas que passavam por ali. Na prática, o diagrama congela em uma versão antiga e passa a divergir do modelo que deveria documentar — o pior resultado possível para um artefato de especificação.

Soma-se a isso a ausência de garantia de fidelidade. Como o desenho é feito em um arquivo separado da base de dados, nada impede que omita uma conexão existente ou apresente uma conexão inexistente. Não há verificação possível: o diagrama é uma segunda fonte da verdade, e uma fonte não auditável.

1.3 Estado da arte: o que já existe e onde para

A geração automática de RES não é território virgem. O trabalho mais próximo é o comando `otoole viz res`, parte do `otoole` — a suíte oficial de ferramentas do ecossistema OSeMOSYS (BARNES; USHER, 2023). Ele monta um grafo dirigido com a biblioteca `networkx` a partir dos parâmetros `InputActivityRatio` e `OutputActivityRatio` do modelo e o desenha com o motor `dot` do Graphviz, configurado com `rankdir=LR`, `splines=ortho` e `concentrate=true`, exportando uma imagem estática e um arquivo `.graphml`.

Essa é a peça de arte prévia mais relevante, e ela resolve o custo de produção. Suas limitações, verificadas diretamente no código-fonte do módulo `otoole/visualise/res.py`, são o que delimita o espaço deste trabalho:

- **A coluna é inferida, não declarada.** O `dot` atribui a camada de cada nó a partir da topologia do grafo, minimizando o comprimento das arestas. O estágio energético — informação semântica que o modelador conhece e que define a leitura "primário → secundário → final → útil → demanda" — não é entrada do algoritmo. Um nó pode ser deslocado de coluna por conveniência do *layout*, quebrando a leitura setorial.
- **Os combustíveis não são entidades visuais de primeira classe.** No código, nós de combustível recebem `height: 0.1`, `width: 0.1` e rótulo vazio: são pontos minúsculos e sem identificação. A leitura do diagrama passa a depender inteiramente dos nomes das tecnologias.
- **A saída é uma imagem.** Não há clique, destaque, filtro ou rastreamento.
- **A extração de subgrafos é demanda aberta e não atendida.** A *issue* 14 do repositório `otoole` solicita exatamente isso — caminho mais curto entre dois nós, todos os predecessores de um nó, tudo que se conecta a um combustível — e permanece aberta desde outubro de 2019, sem implementação. Demanda equivalente aparece na *issue* 102 do projeto `tz-osemosys`, que pede auto-geração de RES e também segue aberta.

1.4 Limitações das visualizações de plataformas de BI

A alternativa de usar um visual de Sankey ou de rede pronto do Power BI falha por três motivos estruturais. Primeiro, o diagrama de Sankey codifica espessura por volume, não

topologia: o RES precisa mostrar quais rotas existem, não quanto flui por elas — e, na fase de especificação, os volumes sequer são conhecidos. Segundo, o eixo horizontal do Sankey é derivado do grafo, e não do estágio energético, reproduzindo o mesmo defeito do `dot`; visuais de rede vão além e adotam *layout* por força dirigida, que abandona por completo a noção de coluna. Terceiro, fluxos reversos não são tratados: quando uma tecnologia consome um combustível de estágio posterior e devolve para um estágio anterior — situação legítima e frequente, como a termelétrica a gás que consome gás processado do estágio 3 e produz eletricidade de geração no estágio 2 — diagramas de Sankey convencionais produzem sobreposição ilegível ou rejeitam o caso como ciclo.

Resta ainda a ausência de interrogabilidade. Mesmo um diagrama bem desenhado, acima de aproximadamente 50 nós, torna-se um emaranhado. A pergunta que o modelador realmente faz — "mostre tudo que depende deste combustível" — não tem resposta em uma imagem estática. Há trabalho acadêmico em visualização interativa de sistemas energéticos (ALEMA-SOOM et al., 2016), mas voltado à exploração de grandezas, como séries temporais, distribuição espacial e emissões, e não à especificação topológica do modelo; além de existir como protótipo de pesquisa, fora do ferramental de trabalho do modelador.

1.5 Formulação do problema técnico

Reduzido à sua essência, o problema é um caso restrito de desenho de grafo em camadas (*layered graph drawing*), classe formalizada por Sugiyama, Tagawa e Toda (1981) e implementada em ferramentas como o `dot` (GANSNER et al., 1993). As diferenças que tornam o caso do RES mais difícil são três:

1. a atribuição de camadas é imposta pelos dados, não otimizada pelo algoritmo: o estágio do combustível é semântico e não pode ser alterado pelo *layout*;
2. existem arestas retrógradas legítimas, que precisam ser desenhadas e não removidas;
3. a ferramenta de destino não é um motor de grafos, e sim o Power BI, cujas primitivas de cálculo são a linguagem M — funcional, orientada a tabelas, sem estruturas mutáveis e sem laços imperativos — e cuja camada de renderização acessível é a gramática declarativa Vega, por meio do conector Deneb.

A lacuna que o projeto ataca, portanto, não é gerar o RES automaticamente — isso o `otoole` já faz. É gerar um RES em que a coluna seja imposta pelo dado semântico, em que os combustíveis sejam eixos rotulados de primeira classe, em que fluxos reversos sejam roteados em vez de rejeitados, e em que o resultado seja interrogável; tudo isso dentro da plataforma de BI em que o restante da análise já vive.

2 Soluções Propostas

2.1 Arquitetura em duas camadas

A decisão arquitetural central foi abandonar os visuais de Sankey prontos e construir um motor próprio, dividido em duas camadas com contrato explícito entre elas, conforme a Tabela 1.

Camada	Tecnologia	Responsabilidade
Back-end geométrico	Power Query / linguagem M	Ler a topologia; alocar coordenadas absolutas; resolver colisões; classificar arestas
Front-end de renderização	Deneb / Vega v5	Desenhar coordenadas pré-calculadas com primitivas vetoriais; gerenciar interação

Tabela 1 — Divisão de responsabilidades entre as duas camadas da solução.

O contrato é uma única tabela achatada, `energy_view`, em que cada linha já carrega tudo o que o renderizador precisa. O Vega não calcula posição: apenas desenha. Essa separação foi o que tornou o problema tratável, pois permitiu desenvolver, testar e depurar cada camada isoladamente.

A entrada do sistema são duas abas de planilha, deliberadamente mínimas. A aba `Fuels` declara `fuel`, `stage` e `color`; a aba `Techs` declara `tech`, `fuel_in` e `fuel_out`. Múltiplas entradas ou saídas de uma mesma tecnologia são expressas como múltiplas linhas com o mesmo identificador de tecnologia — decisão que mantém a planilha plana e editável por quem não programa, conforme ilustra a Figura 2.

(a) aba Fuels — fuel, stage, color

fuel	stage	color
OIL	1	#333333
NGS1	1	#d95f02
COA	1	#666666
CAN	1	#1b9e77
COR	1	#e7298a
OSE	1	#8c564b
BIO	1	#1b9e77
URN	1	#377eb8
RFD	2	#333333
ELC1	2	#7570b3
DIE1	3	#333333
GAS1	3	#333333
HFO	3	#333333
ODR	3	#333333
NGS2	3	#d95f02

(b) aba Techs — tech, fuel_in, fuel_out

PROETA1	CAN	EAN
PROETA1		EHY
PROETA1		BAG
PROETA2	COR	EAN
PROETA2		EHY
PROBDI	OSE	BDI
MIXGAS	EAN	GAS2
MIXGAS	GAS1	GAS2
MIXDIE	BDI	DIE2
MIXDIE	DIE1	DIE2
PROSTH		STH
PWRBIO	BIO	ELC1
PWRBAG	BAG	ELC1
PWRNUC	URN	ELC1
PWRCOA	COA	ELC1
PWRNGS1	NGS2	ELC1

Figura 2 — Estrutura de entrada do sistema. Em (b), as três linhas destacadas declaram as três saídas da tecnologia `PROETA1`.

2.2 Pipeline de transformação

O *back-end* foi decomposto em nove consultas encadeadas, cada uma com responsabilidade única e testável, totalizando 556 linhas de linguagem M. O encadeamento aparece na Figura 3: o parâmetro `CaminhoArquivo` alimenta as duas leituras cruas (`Input_Tech` e `Input_Fuel`), que passam pelas consultas de tipagem (`tech_map` e `fuel_map`) antes de convergirem em `FuelLayout` e `TechLayout`, onde toda a geometria é resolvida; `energy_view` achata o resultado para o visual.

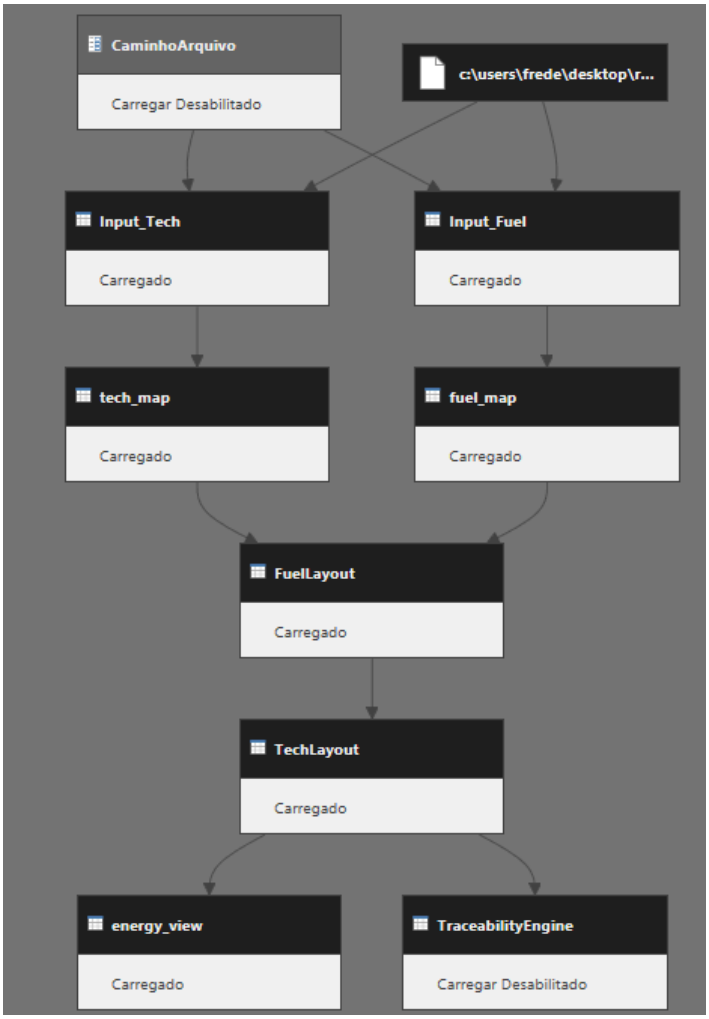


Figura 3 — Grafo de dependências entre as nove consultas do *pipeline*.

2.3 Algoritmos de layout

2.3.1 Eixo horizontal dos combustíveis: look-ahead de densidade

A abordagem ingênua — colunas de largura fixa por estágio — produz diagramas desbalanceados: estágios com poucos combustíveis desperdiçam espaço, e estágios densos ficam comprimidos. A solução implementada calcula a largura de cada faixa em função da densidade do próprio estágio e da do estágio seguinte:

```
LarguraDesteStage =
    GapBasico + ( List.Max({ ContagemDesteStage, ContagemDoProximoStage }) *
PixelsPorItem )
```

As posições finais são obtidas por soma acumulada dessas larguras. O *look-ahead* é o que evita que as linhas que saem de um estágio denso colidam ao entrar no próximo: reserva-se espaço para o pior caso local antes de ele ocorrer. Dentro de cada faixa, os combustíveis são distribuídos por um desvio normalizado em torno do centro, garantindo que nenhum par de colunas verticais se sobreponha.

2.3.2 Eixo horizontal das tecnologias: paredes geométricas e fluxos reversos

Uma tecnologia não pertence a um estágio: ela vive entre estágios. Para posicioná-la, cada estágio é reduzido a duas paredes — o menor e o maior valor horizontal entre seus combustíveis. Para cada tecnologia calculam-se `Max_Stage_In` e `Min_Stage_Out`, e a posição é resolvida por três casos. Quando há entrada e saída em fluxo direto, a caixa é centrada entre a parede direita da entrada e a parede esquerda do estágio seguinte; quando `Max_Stage_In` é maior ou igual a `Min_Stage_Out`, trata-se de fluxo reverso e as referências espaciais são invertidas. Quando há apenas entrada — tecnologias de demanda ou exportação — a caixa é posicionada à direita da origem; quando há apenas saída — extração, importação e fontes renováveis sem insumo, 25 tecnologias no sistema de validação — é posicionada à esquerda do destino.

A classificação de fluxo reverso é derivada nesta etapa e propagada até o Vega, que aplica regras de desenho distintas. No sistema de validação, quatro arestas são fluxos reversos — `PWRBAG`, `PWRHFO`, `PWRNGS1` e `PWRNGS2` —, todas roteadas automaticamente sem qualquer declaração do usuário; três delas aparecem no recorte da Figura 4.

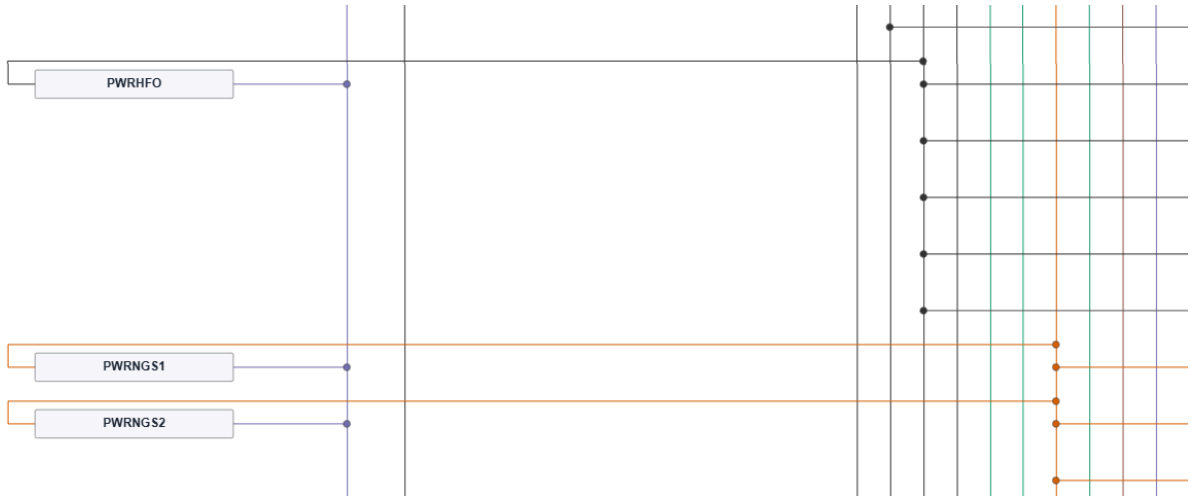


Figura 4 — Roteamento automático de fluxos reversos nas termelétricas `PWRHFO`, `PWRNGS1` e `PWRNGS2`.

2.3.3 Eixo vertical: raias elásticas e nós fantasmas

Este é o principal resultado técnico do projeto. Empilhar as tecnologias verticalmente por raia do combustível de origem resolve a colisão entre caixas, mas não resolve a colisão entre caixas e linhas. Uma conexão de longo alcance — por exemplo, um combustível do estágio 1 consumido por uma demanda do estágio 5 — atravessa horizontalmente várias colunas e passa por cima de caixas de tecnologia alheias, produzindo um cruzamento ilegível.

Em vez de rotear a linha desviando dos obstáculos, o que exigiria busca em grade e seria inviável em M puro, o algoritmo reserva espaço antes de desenhar. Para cada conexão longa, gera-se um conjunto de nós fantasmas: entidades invisíveis, uma para cada coluna ocupada que a linha cruza.

```
GeraGhostsGeometricos = Table.AddColumn(FiltraLongHaul, "GhostGrid", each
    let
        Start = List.Min({[x_tech], [x_fuel_out]}),
        End   = List.Max({[x_tech], [x_fuel_out]}),
        GridsNoCaminho = List.Select(ListaGridsOcupados,
                                     (g) => g > (Start + 25) and g < (End - 25))
    in
        GridsNoCaminho
)
```

Os fantasmas são contabilizados como se fossem caixas reais no cálculo de densidade local, mas não são desenhados: eles são descartados antes da renderização, e sua única consequência é aritmética. A altura reservada para cada raia passa a ser o pior caso ao longo de toda a sua extensão horizontal, e a posição vertical de base de cada raia é obtida por soma acumulada dessas alturas. O efeito é que a faixa vertical de uma raia deixa de ser dimensionada apenas pelo empilhamento local de suas caixas e passa a acompanhar também o percurso das conexões longas que dela partem; como o empilhamento é acumulado, toda raia seguinte é deslocada para baixo, ampliando a folga vertical ao longo desse percurso.

A Figura 5 resume o mecanismo em três etapas. Em (a), sem a reserva, a raia dispõe de uma única linha: T1, T2 e T3 a compartilham, e a conexão longa que parte de T1 passa por cima de T2 e T3. Em (b), cada coluna ocupada que a conexão cruza recebe um nó fantasma; o fantasma toma a linha por onde a conexão precisa passar, empurra as caixas cruzadas para a linha seguinte e abre a folga necessária. Em (c), o desenho se acomoda: T1 permanece sozinha na sua linha, com o percurso desobstruído, e as caixas deslocadas passam a ocupar a raia seguinte.

A lista de colunas ocupadas é global, e não por raia: o fantasma sinaliza que a conexão transita sobre uma região do desenho onde existem caixas, independentemente de a quem elas pertençam. O ajuste é obtido por dimensionamento — a raia passa a reservar tantas linhas

quantas o percurso exige —, e não por desvio de traçado, e é isso que permite expressá-lo apenas como transformações de tabela, sem busca em grade.

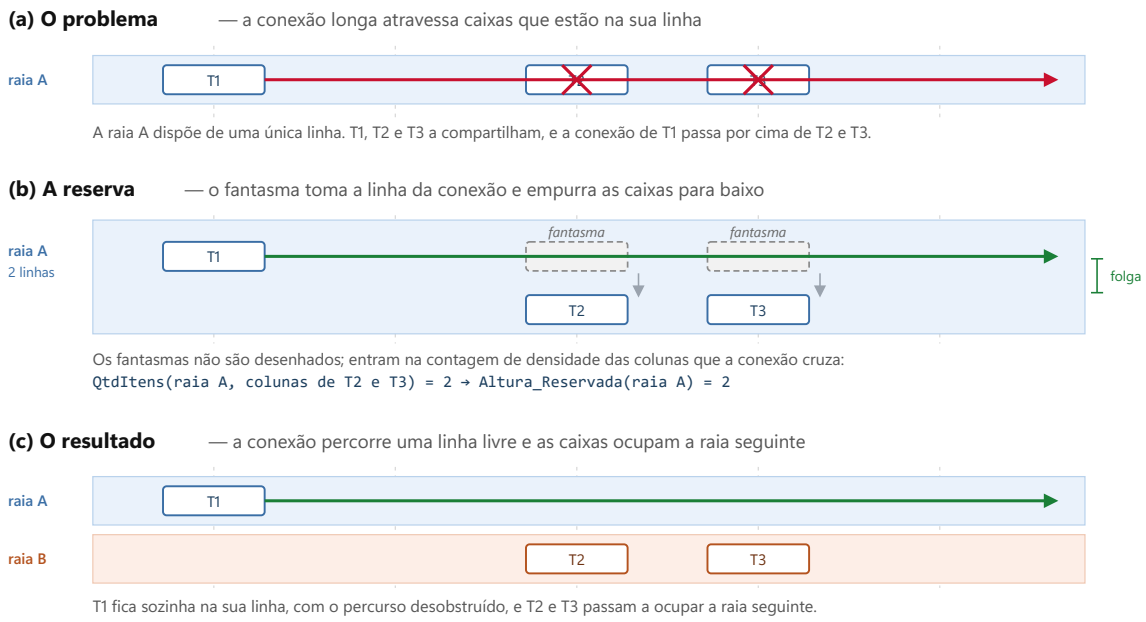


Figura 5 — Algoritmo de raias elásticas com nós fantasmas.

Trata-se de uma troca deliberada: aceita-se um diagrama um pouco mais alto em favor de um algoritmo determinístico, sem busca e expressável apenas como transformações de tabela. Duas correções foram necessárias para estabilizar o algoritmo, ambas documentadas no código: a contagem de densidade passou a usar deduplicação por tecnologia, para que uma tecnologia com três saídas não infle a altura da raia como se fossem três caixas; e o índice vertical passou a ser atribuído sobre a lista de tecnologias distintas e depois reassociado por junção, garantindo que todas as arestas de uma mesma tecnologia apontem para a mesma caixa.

2.3.4 Lógica de hub e herança de cor

Nem toda aresta do RES tem tecnologia associada — há ligações diretas entre combustíveis. O sistema gera identificadores sintéticos para esses casos, agrupando por origem, e os marca com uma bandeira que instrui o Vega a desenhar apenas a linha, sem a caixa. Isso mantém uma única representação interna para todas as arestas, sem ramificação especial no renderizador.

Há também um mecanismo de herança automática de cor: combustíveis de demanda herdam a cor do combustível que os origina, de modo que uma cadeia inteira permanece cromaticamente coerente sem que o usuário precise preencher a coluna de cor para dezenas de nós. No sistema de validação, 46 dos 70 combustíveis são nós de demanda beneficiados por essa regra.

2.4 Camada de interação

A especificação Vega, com aproximadamente 1.030 linhas, organiza-se em nove conjuntos de dados — o conjunto único vindo do Power Query e oito derivações dele, obtidas por filtro e fórmula — e dezesseis marcas de desenho.

Cada elemento clicável é desenhado duas vezes: uma marca visual fina, de 2 pixels, e uma marca de captura praticamente invisível, de 45 pixels — 20 pixels no caso dos fluxos reversos —, com opacidade de 0,001. O usuário vê uma linha fina e elegante, mas dispõe de uma área de clique vinte vezes maior. Sem esse recurso, acertar uma linha de 2 pixels com o cursor em um diagrama denso seria inviável na prática.

Cinco sinais reativos implementam a interrogação do diagrama, descritos na Tabela 2. Todos respondem também ao duplo clique no fundo, restaurando o estado integral. O destaque opera por opacidade, e não por remoção: o contexto permanece visível, atenuado, o que preserva a leitura global enquanto se examina um detalhe.

Sinal	Evento	Efeito
`sel_fuel_color`	clique no combustível	Isola a coluna e todos os fluxos de mesma cor
`sel_tech_color`	clique na tecnologia	Destaca a tecnologia e seus fluxos de entrada e saída
`sel_line_color`	clique na entrada	Ilumina os fluxos horizontais de entrada
`sel_line_bf1_color`	clique no fluxo reverso	Destaca rotas retrógradas
`sel_line_out_color`	clique na saída	Destaca fluxos de saída

Tabela 2 — Sinais reativos que implementam o destaque interativo.

As Figuras 6, 7 e 8 demonstram o mecanismo sobre a mesma região do diagrama. A Figura 6 mostra o estado íntegro, sem seleção: todos os combustíveis e tecnologias em opacidade plena.

A Figura 7 registra o clique sobre o combustível `BIO` e evidencia a semântica do agrupamento. O que acende não é apenas a coluna clicada, mas todo o **grupo cromático**: como `BIO` e `CAN` compartilham a mesma cor no cadastro, as duas colunas se iluminam juntas, e com elas as quatro demandas setoriais de biomassa, a termelétrica `PWRBIO` e a rota de etanol `PROETA1` — esta última alimentada por `CAN`, e não por `BIO`. O comportamento é deliberado: a cor no cadastro expressa a classe do vetor energético, de modo que a seleção responde à pergunta “onde esta classe de energia é usada?”, e não apenas “o que consome exatamente este nó?”.

A Figura 8 mostra o contraste. O carvão tem cor exclusiva no cadastro, e por isso o clique sobre `COA` isola estritamente a sua própria cadeia; a cadeia da biomassa, destacada na figura anterior, permanece visível ao fundo em opacidade reduzida. É essa atenuação, em vez da remoção, que preserva a noção de onde a cadeia isolada se situa no todo.

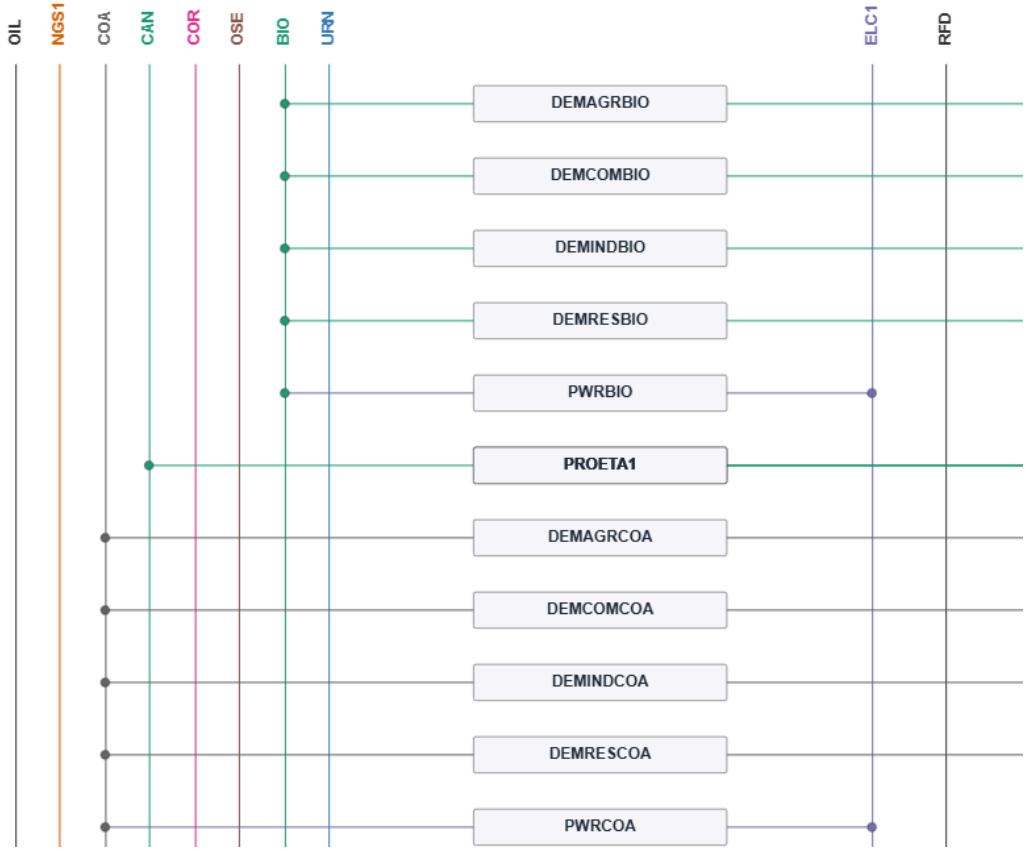


Figura 6 — Estado íntegro, sem seleção ativa.

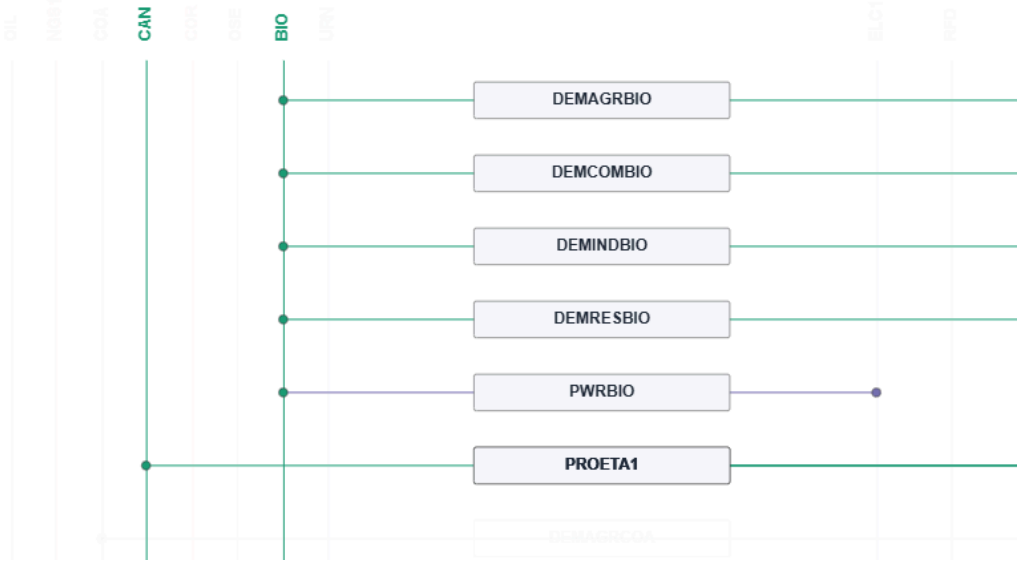


Figura 7 — Seleção do combustível `BIO`: a cadeia da biomassa é isolada.



Figura 8 — Seleção do combustível `COA`: a cadeia do carvão é isolada e a da biomassa, atenuada.

Fluxos retrógrados são desenhados por quatro marcas coordenadas, apoiadas em coordenadas auxiliares calculadas ainda no Power Query: a altura da ponte que transpõe as caixas e o ponto da curva de retorno.

2.5 Motor de rastreabilidade

Para responder à pergunta sobre quais nós dependem de um dado nó, foi implementada busca em grafo — ancestrais e descendentes — em M puro. Como a linguagem não oferece estruturas mutáveis, a travessia usa iteração funcional com estado em registro sobre mapas de adjacência pré-construídos, o que converte a busca de vizinhos de uma varredura linear de tabela para um acesso direto por chave. A recursão é limitada a vinte níveis como salvaguarda contra ciclos, e as saídas são cadeias delimitadas, diretamente consumíveis por um teste de subcadeia no Vega.

O motor está implementado e funcional, porém desativado por padrão no *pipeline* final — condição visível na Figura 3, em que a consulta `TraceabilityEngine` aparece marcada como "Carregar Desabilitado". Sua materialização impacta sensivelmente o tempo de atualização do modelo, e o destaque por cor já cobre a maior parte dos casos de uso exploratório. A decisão, seus motivos e o caminho de reativação estão documentados no tutorial do desenvolvedor. Trata-se de compromisso explícito, não de código morto.

2.6 Engenharia de processo e validação

Um problema recorrente em projetos de BI é que o código fica preso dentro do binário. Um arquivo `.pbix` não é diferenciável, não é revisável e não tem histórico útil, o que inviabiliza revisão por pares. A solução adotada foi externalizar todo o código como texto versionado, com mapeamento de um para um entre arquivo e objeto do Power BI: nove arquivos de consulta M e um arquivo de especificação Vega, acompanhados de um protocolo de sincronização e de um script que reconstrói o pacote completo para auditoria de divergência. O repositório passa a ser a fonte da verdade; o `.pbix` é apenas o ambiente de execução.

Para a validação, foi desenvolvido um gerador de bases com topologias aleatórias, incluindo casos patológicos: combustíveis órfãos, tecnologias sem entrada, conexões de altíssimo alcance e múltiplos fluxos reversos simultâneos. A ferramenta foi exercitada contra essas bases para verificar que o algoritmo de *layout* não depende de propriedades particulares do sistema energético usado no desenvolvimento.

3 Resultados Alcançados

3.1 Produto entregue

Foi entregue uma ferramenta funcional, documentada e versionada que gera diagramas RES completos e interativos a partir de duas tabelas, sem nenhuma coordenada desenhada manualmente. As características quantitativas do sistema de validação estão resumidas na Tabela 3.

O diagrama gerado é grande o bastante para que sua reprodução integral em página impressa seja ilegível: são 70 colunas de combustível e 101 caixas de tecnologia distribuídas ao longo de milhares de pixels em ambos os eixos. Essa extensão é consequência direta do objetivo — representar o sistema energético inteiro, e não um recorte dele — e é também a razão pela qual a ferramenta precisa ser interativa: o zoom e o destaque por seleção substituem a visão global que uma imagem estática não consegue entregar. Por isso, todas as figuras deste relatório que mostram a saída são recortes, e há um compromisso direto entre a abrangência do recorte e a legibilidade dos rótulos. A Figura 9 ocupa o extremo da abrangência: é um panorama que serve para dimensionar a densidade e a regularidade do traçado, não para ler nomes de tecnologia — estes aparecem em escala plena nas figuras de detalhe das seções anteriores.

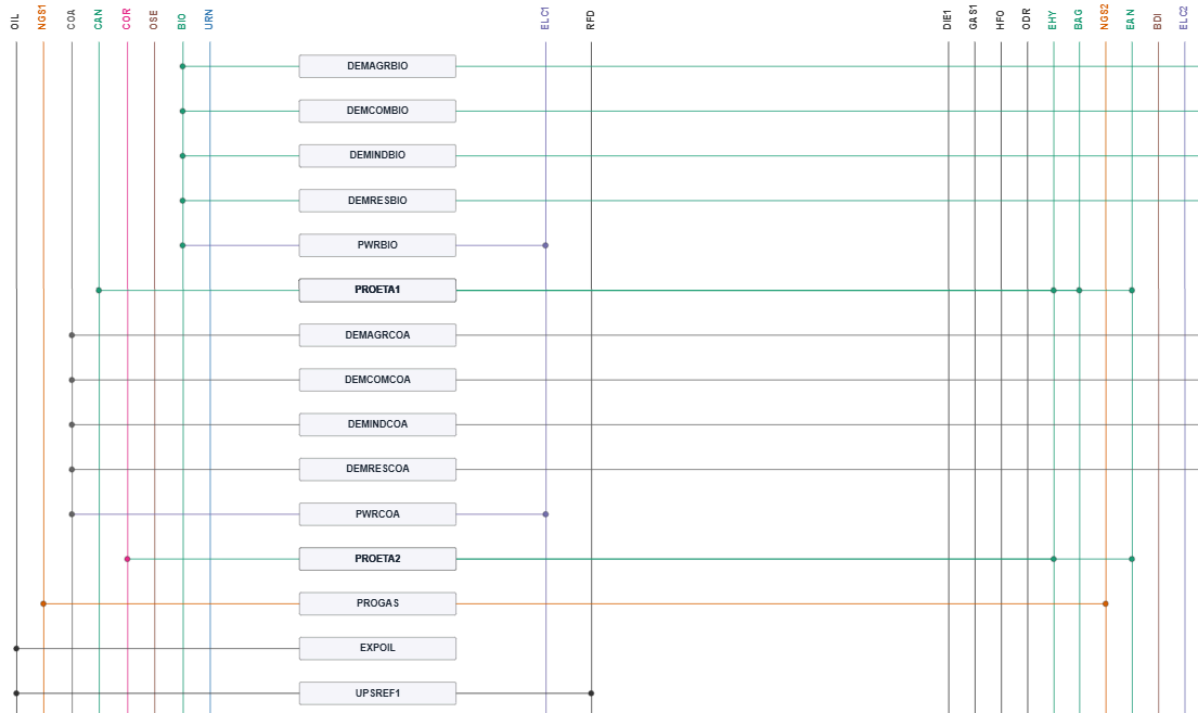


Figura 9 — Panorama do diagrama gerado automaticamente, abrangendo os combustíveis primários e secundários e parte das rotas de conversão. Reproduzido em escala reduzida para evidenciar a extensão do traçado.

Métrica	Valor
Combustíveis (nós verticais)	70
Estágios energéticos	5
Distribuição de combustíveis por estágio	8 / 2 / 10 / 4 / 46
Tecnologias distintas	101
Arestas declaradas	106
Tecnologias-fonte (apenas saída)	25
Tecnologias com múltiplas saídas	2
Tecnologias com múltiplas entradas	2
Fluxos reversos detectados e roteados	4
Cores semânticas distintas	10
Coordenadas desenhadas manualmente	0

Tabela 3 — Características do sistema energético usado na validação da ferramenta.

Do ponto de vista de esforço de produção, o mesmo diagrama desenhado à mão representa trabalho da ordem de dias; com a ferramenta, a alteração de uma linha na planilha e um comando de atualização reconstroem o diagrama inteiro — reposicionamento, roteamento e cores — em segundos. Mais relevante do que o ganho na primeira produção é o ganho na manutenção: o custo marginal de manter o diagrama sincronizado com o modelo cai a praticamente zero, o que elimina a causa raiz do problema diagnosticado.

3.2 Contribuições técnicas

A combinação de *look-ahead* de densidade no eixo horizontal com raias elásticas e nós fantasmas no eixo vertical resolve o problema de sobreposição sem busca e sem iteração de otimização, apenas com transformações de tabela. É isso que torna possível executá-lo dentro do motor do Power Query, um ambiente não projetado para algoritmos de grafo.

A distinção em relação à arte prévia merece enunciado preciso. A geração automática de RES já existe, oferecida pelo ecossistema oficial do OSeMOSYS. A diferença deste trabalho está em quem decide a coluna. Em *layouts* do tipo `dot`, a camada de cada nó é derivada da topologia pelo algoritmo; aqui, ela é imposta pelo campo de estágio declarado nos dados, e o algoritmo trabalha sob essa restrição em vez de otimizá-la. Essa inversão é o que preserva a leitura semântica do RES, e é também o que cria o problema de sobreposição que os nós fantasmas resolvem — um motor livre para mover nós entre camadas simplesmente não enfrenta esse caso. Na revisão realizada, não foi identificada implementação equivalente, que reúna camada imposta pelos dados, combustíveis rotulados como eixos de primeira classe, roteamento explícito de fluxos reversos e interatividade, nativa em plataforma de BI.

Os fluxos retroalimentados são detectados automaticamente pela comparação de estágios, sem declaração do usuário, e roteados por conjunto dedicado de regras de desenho com coordenadas de ponte — justamente a classe de caso que os visuais de mercado rejeitam ou renderizam de forma ilegível.

O diagrama, por fim, deixou de ser imagem e passou a ser interface. Cabe registrar que a capacidade de extrair e destacar o subgrafo conectado a um nó é precisamente o que a *issue* 14 do repositório `otoole` solicita desde outubro de 2019, sem implementação até o momento; o motor de rastreabilidade deste projeto endereça, ainda que em outro ambiente, uma demanda reconhecidamente em aberto na ferramenta de referência do ecossistema.

O algoritmo não contém nenhuma regra específica do sistema energético brasileiro nem do vocabulário utilizado: opera sobre a forma do grafo. A validação com bases sintéticas aleatórias confirma que qualquer matriz origem–destino no formato padronizado é renderizada sem reconfiguração, o que abre aplicação para fluxos de materiais, cadeias produtivas e processos industriais.

3.3 Produção documental

O projeto produziu documentação em três níveis, além do código comentado: um guia do usuário, voltado ao modelador energético, cobrindo a estrutura das planilhas, a atualização de dados e a leitura e interação com o diagrama; um tutorial do desenvolvedor, voltado ao futuro mantenedor, dissecando cada consulta, a lógica de *hub*, o cálculo de paredes, os nós fantasmas, o motor de rastreabilidade e os sinais do Vega; e um resumo executivo de resultados e arquitetura.

Somam-se a estes o arquivo de descrição do repositório e o protocolo de sincronização entre o repositório e o Power BI.

3.4 Dificuldades encontradas

A principal dificuldade foi a limitação da linguagem M para algoritmos de grafo. M é funcional e sem estado mutável: não há vetores indexados para escrita, não há laços imperativos, não há recursão convencional eficiente. Todo o algoritmo de *layout* precisou ser reexpresso como sequência de agrupamentos, acumulações, junções e indexações de tabela. Essa restrição, contudo, produziu a solução dos nós fantasmas, que é resposta direta a ela: não sendo possível rotear desviando de obstáculos, optou-se por reservar espaço antes de desenhar. A limitação da ferramenta determinou a forma da contribuição técnica.

A segunda dificuldade foi o desempenho do motor de rastreabilidade. A implementação por mapas de adjacência reduziu drasticamente o custo da busca, mas a materialização das colunas de caminho ainda onera a atualização do modelo, o que motivou desativá-la por padrão.

A terceira envolveu a estabilidade do cálculo do eixo vertical, com duas classes de defeito: a inflação espúria da altura da raia quando uma tecnologia possuía múltiplas saídas, e a dispersão de uma mesma tecnologia por posições verticais distintas. Ambas foram resolvidas pela deduplicação prévia à indexação, com reassociação posterior por junção.

Por fim, o Power BI não expõe interface de escrita para consultas M ou especificações Deneb. O ciclo de edição exige transporte manual do arquivo-texto para o editor da ferramenta, o que motivou a criação do protocolo de sincronização e do script de auditoria, destinados a tornar esse ciclo confiável e a permitir detectar divergências entre o ambiente de execução e a fonte.

3.5 Trabalhos futuros

1. **Parametrização via interface.** Externalizar as constantes de *layout* para campos de parâmetro do Power BI, permitindo ajuste fino sem editar código.
2. **Reativação otimizada da rastreabilidade.** Avaliar cálculo incremental ou pré-computação externa dos caminhos, evitando o gargalo na atualização.
3. **Dimensionamento por volume.** Acrescentar coluna opcional de fluxo e modular a espessura das linhas, unindo a expressividade topológica do RES à quantitativa do Sankey.
4. **Exportação vetorial.** Gerar saída em resolução de publicação, para uso direto em artigos e relatórios técnicos.
5. **Validação semântica da base.** Aproveitar o grafo já construído para emitir diagnósticos automáticos — combustível órfão, tecnologia inalcançável, demanda não atendida, ciclo não declarado —, transformando a ferramenta de visualizador em verificador de consistência do modelo.

6. **Aplicação a outros domínios.** Testar a ferramenta sobre fluxos de materiais e cadeias produtivas, domínios com a mesma estrutura de grafo em camadas.

3.6 Conclusão

O projeto entregou aquilo a que se propôs: um gerador automático, dinâmico e interativo de diagramas de fluxo de energia, que elimina o gargalo do desenho manual e, sobretudo, o custo de manutenção que fazia o diagrama divergir do modelo. O resultado técnico central — o algoritmo de raias elásticas com nós fantasmas — é resposta original às restrições de um ambiente de cálculo funcional sem estado mutável, e é generalizável a qualquer grafo em camadas com restrição semântica de camada. A ferramenta é escalável a qualquer base relacional que siga a matriz origem–destino padronizada, está documentada em três níveis e tem seu código versionado como texto auditável, pronto para continuidade.

Agradecimentos

Este trabalho foi desenvolvido no âmbito do Programa Institucional de Bolsas de Iniciação Científica em Desenvolvimento Tecnológico e Inovação (PIBITI), com bolsa concedida pelo Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) por meio do edital PIBITI/CNPq — PIBINOVA/PDI/UFF 2025-2026, da Agência de Inovação (AGIR/PROPPI) da Universidade Federal Fluminense. O autor agradece ao professor Tiago Abud pela orientação ao longo de todas as fases do projeto.

Declaração de uso de Inteligência Artificial Generativa

Em atendimento à Portaria CNPq nº 2.664, de 6 de março de 2026, que institui a Política de Integridade na Atividade Científica, declara-se o uso de ferramentas de Inteligência Artificial Generativa em todas as fases deste trabalho, discriminado a seguir.

Ferramentas utilizadas. Google Gemini (Google) e Claude (Anthropic), ambos operados diretamente pelo bolsista.

Desenvolvimento da ferramenta. Os dois assistentes foram empregados ao longo de toda a construção e dos testes do DDFE: discussão das alternativas de algoritmo para os eixos horizontal e vertical; escrita e depuração das consultas em linguagem M e da especificação Vega; investigação de erros; e verificação do comportamento do *layout* sobre as bases de teste, incluindo as topologias sintéticas de esforço.

Elaboração deste relatório. O assistente Claude foi utilizado na redação e revisão do texto a partir da documentação técnica e do código-fonte do projeto; na verificação dos dados

quantitativos contra as planilhas de entrada e as consultas em M; na elaboração da Figura 5 e na composição da Figura 2 a partir de capturas de tela do autor; e na diagramação do documento.

Responsabilidade. A condução do projeto, a definição dos objetivos e requisitos, as decisões técnicas, a validação dos resultados e a revisão crítica de todo o material gerado couberam ao bolsista, que responde integralmente pelo conteúdo apresentado. Nenhum conteúdo produzido pelas ferramentas foi incorporado sem verificação.

Referências

ALEMASOOM, H.; SAMAVATI, F.; BRODLIE, K.; LAYZELL, D. EnergyViz: an interactive system for visualization of energy systems. **The Visual Computer**, v. 32, n. 3, p. 403–413, 2016.

BARNES, T.; USHER, W. otoole: OSeMOSYS Tools for Energy Work. **Journal of Open Source Software**, v. 8, n. 92, art. 5511, 2023. DOI: 10.21105/joss.05511.

EMPRESA DE PESQUISA ENERGÉTICA. **Balanço Energético Nacional**. Rio de Janeiro: EPE. Disponível em: https://www.epe.gov.br/pt/publicacoes-dados-abertos/publicacoes/balanco-energetico-nacional.

GANSNER, E. R.; KOUTSOFIOS, E.; NORTH, S. C.; VO, K.-P. A technique for drawing directed graphs. **IEEE Transactions on Software Engineering**, v. 19, n. 3, p. 214–230, 1993.

HOWELLS, M. et al. OSeMOSYS: The Open Source Energy Modeling System — An introduction to its ethos, structure and development. **Energy Policy**, v. 39, n. 10, p. 5850–5870, 2011.

LOULOU, R.; REMME, U.; KANUDIA, A.; LEHTILÄ, A.; GOLDSTEIN, G. **Documentation for the TIMES Model**. Energy Technology Systems Analysis Programme (ETSAP), 2005.

MARSH-PATRICK, D. **Deneb: Declarative Visualization in Power BI**. Documentação oficial. Disponível em: https://deneb-viz.github.io/.

MICROSOFT. **Power Query M formula language reference**. Disponível em: https://learn.microsoft.com/en-us/powerquery-m/.

OSEMOSYS. **Create a model in OSeMOSYS — Reference Energy System**. Documentação oficial. Disponível em: https://osemosys.readthedocs.io/.

OSEMOSYS/OTOOLE. **Issue 14 — otoole viz res: Visualise subgraph of the reference energy system**. Repositório GitHub, aberta em out. 2019. Disponível em: https://github.com/OSeMOSYS/otoole/issues/14.

OTOOLE. **Core Functionality**. Documentação oficial. Disponível em: https://otoole.readthedocs.io/en/latest/functionality.html.

RIEHMANN, P.; HANFLER, M.; FROEHLICH, B. Interactive Sankey diagrams. In: **IEEE Symposium on Information Visualization (InfoVis)**, 2005, p. 233–240.

SATYANARAYAN, A.; MORITZ, D.; WONGSUPHASAWAT, K.; HEER, J. Vega-Lite: A grammar of interactive graphics. **IEEE Transactions on Visualization and Computer Graphics**, v. 23, n. 1, p. 341–350, 2017.

SATYANARAYAN, A.; RUSSELL, R.; HOFFSWELL, J.; HEER, J. Reactive Vega: A streaming dataflow architecture for declarative interactive visualization. **IEEE Transactions on Visualization and Computer Graphics**, v. 22, n. 1, p. 659–668, 2016.

SCHMIDT, M. The Sankey diagram in energy and material flow management. **Journal of Industrial Ecology**, v. 12, n. 1–2, 2008.

SUGIYAMA, K.; TAGAWA, S.; TODA, M. Methods for visual understanding of hierarchical system structures. **IEEE Transactions on Systems, Man, and Cybernetics**, v. 11, n. 2, p. 109–125, 1981.

TRANSITION ZERO. **Issue 102 — ENH: auto-generate a Reference Energy System (RES) diagram**. Repositório tz-osemosys, GitHub. Disponível em: https://github.com/transition-zero/tz-osemosys/issues/102.

UNIVERSIDADE FEDERAL FLUMINENSE. **Edital para submissão de projetos PIBITI/CNPq — PIBINOVA/PDI/UFF 2025-2026**. Agência de Inovação (AGIR/PROPPI), 2025.